G9_Task

Hello,

Please read the following carefully:

As a Software Engineering research team at the Polytechnique Montréal (Canada), we are interested in understanding developers' assessment of coding task difficulty. You will be provided with a series of 12 coding questions. Questions are divided into two parts:

- First, you will be given three instructions representing coding tasks and you will be asked to write, in a few words, what is the underlying task represented by all these instructions. The instructions represent the same tasks, but they vary in wording and information they give. In a nutshell, you have to distill the essence of the task and write it in a few words.

- In a second time, you will be asked to assess how difficult implementing this ***task*** would be. Note we ask the assessment based on the task you will have worded in the previous part, not based on any of the instructions presented earlier. First, you will be asked your subjective judgment on a scale from Very Easy to Very Difficult. Then, to make judging the difficulty more relatable, we adopt a similar approach as what is used in agile development and planning poker, i.e. you will be asked to evaluate how long it would take to implement a task (in minutes here). Consider in your assessment the knowledge needed to implement the task, the external information you would need to use, whether you would need the assistance of a LLM such as ChatGPT (and so the time it would take to debug the obtained code) etc. Finally, you will have to explain in a few words why you gave such difficulty ratings.

To give you an idea of the range of tasks you will need to assess, the first section of this questionnaire will be two examples we devised to illustrate the questionnaire.

Completing the task will take about 30 minutes.

Note also that all examples are written using Python 3.10 syntax.

Please make sure to answer all the questions to the best of your ability.

Please note that your identity and personal data will not be disclosed, while we plan to use the aggregated results and anonymized responses as part of a scientific publication. If you have any questions about the questionnaire or our research, please do not hesitate to contact us.

By proceeding, you consent to take part in the study under the conditions described previously.

Thank you,

Florian Tambon (florian-2.tambon@polymtl.ca)
Cyrine Zid (cyrine.zid@polymtl.ca)
Amin Nikanjam (amin.nikanjam@polymtl.ca)
Foutse Khomh (foutse.khomh@polymtl.ca)
Giuliano Antoniol (giuliano.antoniol@polymtl.ca)

---

* Indique une question obligatoire

1. **Please enter your prolific ID :** *

Example of tasks

As we ask of you some estimation of difficulty and the tasks dealt in this questionnaire can be a bit different from what you can be used to as a developer, we give you examples of some tasks along with some assessments.

Note that this assessment is **our assessment**. You may have a different opinion based on your background. The idea of those examples is to give an extreme range of what type of tasks you can expect in the questionnaire.

As a reminder: You will first be given three instructions and asked to explain what is the task is about in a few words. Then, you will have to rate the difficulty of the ***task*** you just described (not accounting for particular instruction). Rating the difficulty is done in two steps: subjective difficulty and approximate time needed. Finally, you will be asked to say why you gave such rating.

Example 1

Instruction 1

```
def subtract(c1, c2):
        """
        Deliver the result of subtracting two complex numbers,
        specified as 'c1' and 'c2', as a complex output.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        """
```

Instruction 2

```
def subtract(c1, c2):
        """
        The method receives two complex numbers 'c1' and 'c2', from which it
        subtracts 'c2.real' from 'c1.real' to determine the real portion, and
        subtracts 'c2.imag' from 'c1.imag' to determine the imaginary portion.
        It constructs and returns a new complex number using these results
        with the 'complex()' function.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        """
```

Instruction 3

```
def subtract(c1, c2):
        """
        Subtracts two complex numbers.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        >>> complexCalculator = ComplexCalculator()
        >>> complexCalculator.subtract(1+2j, 3+4j)
        (-2-2j)
        """
```

Answer

The task encapsulated by the **subtract**

function

is to compute the subtraction between two complex numbers.

This task has a subjective difficulty of **Very Easy** as assessed by us and would take about **1 min**
to implement. There is no particular difficulty, the task is very straightforward without any corner case, it just requires to know what a complex is, and the standard Python library for it.

Example 2

Context

```python
import numpy as np
class MetricsCalculator2:
    """
    The class provides to calculate Mean Reciprocal Rank (MRR) and Mean Average Precision (MAP)
    based on input data, where MRR measures the ranking quality and MAP measures the average precision.
    """

    def __init__(self):
        pass

    @staticmethod
    def map(data):
        pass
```

Instruction 1

```python
def mrr(data):
        """
        Compute the MRR of the input data. MRR is a widely used evaluation index.
        It is the mean of reciprocal rank.
        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.
        >>> MetricsCalculator2.mrr(([1, 0, 1, 0], 4))
        >>> MetricsCalculator2.mrr(([([1, 0, 1, 0], 4), ([0, 1, 0, 1], 4)])
        1.0, [1.0]
        0.75, [1.0, 0.5]

        """
```

Instruction 2

```python
def mrr(data):
        """
        Assess the Mean Reciprocal Rank (MRR) from the input 'data'.
        MRR assesses the average of reciprocal ranks of outcomes.
        The input structure 'data' must be a tuple consisting of a list of
        binary values and its total count or a list of such tuples.
        Each binary value (0 or 1) represents the correctness of an answer.
        If 'data' is a tuple, the function returns its mean reciprocal rank.
        If 'data' is a list, the function returns the overall average MRR along
        with a list showing reciprocal ranks for each component tuple.

        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.

        """
```

Instruction 3

```
def mrr(data):
        """
        Evaluate the Mean Reciprocal Rank (MRR) from 'data'. This metric calculates the mean of reciprocal
        ranks for correct answers. Input 'data' could be either a tuple containing binary correctness indicators
        and the aggregate count of answers, or multiple tuples of this type in a list. For each tuple, the function
        measures reciprocal ranks by multiplying the binary indicators by reciprocals of their positions, finding the
        earliest nonzero value to establish the rank of the first correct response. If the input 'data' is a tuple,
        the MRR calculated for this specific tuple is returned; if it's a list, the function summarizes by providing
        both the average MRR and a list of individual MRR values for each tuple.

        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.

        """
```

Answer

The task encapsulated by the **mrr** function is to compute the
Mean Reciprocal Rank of binary tuples data.

This task has a subjective difficulty of **Very Hard** as assessed by us and would take about **40 min**
to implement. The difficulty mainly branches from dealing with the
tuple data and how the formula is processed accordingly (with a few corner cases). This can take
additional time if the Mean Reciprocal Rank is not known.

Question 1:

Below are instructions for implementing the function.

Instruction 1

```
def flip_case(string: str) -> str:
    """
    For a given string, flip lowercase
    characters to uppercase and uppercase
    to lowercase.
    >>> flip_case('Hello')
    'hELLO'
    """
```

Instruction 2

```
def flip_case(string: str) -> str:
    """
    Implement the function 'flip_case' which alters
    each lowercase letter in a string to uppercase,
    and each uppercase letter to lowercase.
    """
```

Instruction 3

```
def flip_case(string: str) -> str:
    """
    Create a function called 'flip_case' that accepts
    a parameter 'string'. It should return a string
    where each letter's casing is reversed, turning
    uppercase to lowercase and vice versa. It performs
    this by utilizing the 'map' function with a lambda
    that executes the 'swapcase()' method on each character
    of 'string', and the outcome of the map is combined into
    one string with 'join()' before being returned.
    """
```

2. In a few words, what is the task this function aims to implement? *

3. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

4. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ◯ 5 min
- ◯ 10 min
- ◯ 20 min
- ◯ 40 min
- ◯ > 40 min

5. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 2:

Below are instructions for implementing the function:

Context

```
class SQLGenerator:
    """
    This class generates SQL statements for common operations on a table,
    such as SELECT, INSERT, UPDATE, and DELETE.
    """

    def __init__(self, table_name):
        """
        Initialize the table name.
        :param table_name: str
        """
        self.table_name = table_name

    def select(self, fields=None, condition=None):
        pass

    def insert(self, data):
        pass

    def update(self, data, condition):
        pass

    def delete(self, condition):
        pass

    def select_by_age_range(self, min_age, max_age):
        pass
```

Instruction 1

```
def select_female_under_age(self, age):
    """
    Generates a SQL statement to select females under
    a specified age.
    :param age: int. The specified age.
    :return: str. The generated SQL statement.
    >>> sql.select_female_under_age(30)
    "SELECT * FROM table1 WHERE age < 30 AND gender = 'female';"
    """
```

Instruction 2

```
def select_female_under_age(self, age):
    """
    Construct a SQL command to retrieve females
    younger than a specified age through the
    function 'select_female_under_age'. This
    function generates a filtering condition
    to only include records where the gender
    is 'female' and the age is below the provided
    'age'. Subsequently, another method is used
    to craft a complete SQL command from this
    condition.
    :param age: int. The specified age.
    :return: str. The generated SQL statement.
    """
```

Instruction 3

```
def select_female_under_age(self, age):
    """
    Craft a SQL statement that selects female
    individuals younger than a given age using
    the function 'select_female_under_age'.
    :param age: int. The specified age.
    :return: str. The generated SQL statement.
    """
```

6. In a few words, what is the task this function aims to implement? *

_____

7. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

8. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ◯ 1 min
- ◯ 5 min
- ◯ 10 min
- ◯ 20 min
- ◯ 40 min
- ◯ > 40 min

9. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 3:

Below are instructions for implementing the function:

Instruction 1

```
def fizz_buzz(n: int):
    """
    Return the number of times the digit 7
    appears in integers less than n which
    are divisible by 11 or 13.
    >>> fizz_buzz(50)
    0
    >>> fizz_buzz(78)
    2
    >>> fizz_buzz(79)
    3
    """
```

Instruction 2

```
def fizz_buzz(n: int):
    """
    Write a function named "fizz_buzz" that
    takes an integer ""n"" as its parameter.
    The function initializes a counter ""cnt""
    to 0 and iterates through numbers from 0
    to "n-1". For each number ""i"", it checks
    if ""i"" is divisible by 11 or 13. If it is,
    the function converts ""i"" to a string and
    uses a filter function within a list
    comprehension to count how many "7"s are present
    in the string representation of ""i"". The count
    of "7"s for each valid ""i"" is added to
    the counter ""cnt"". Finally, the function
    returns the total count ""cnt"" as the result.
    """
```

Instruction 3

```
def fizz_buzz(n: int):
    """
    Create a function called 'fizz_buzz'
    that calculates how often the digit '7'
    is found in numbers smaller than 'n',
    which are divisible by 11 or 13.
    """
```

10. In a few words, what is the task this function aims to implement? *

11. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- [ ] Very Easy
- [ ] Easy
- [ ] Not Easy Nor Difficult
- [ ] Difficult
- [ ] Very Difficult

12. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- [ ] 1 min
- [ ] 5 min
- [ ] 10 min
- [ ] 20 min
- [ ] 40 min
- [ ] > 40 min

13. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 4:

Below are instructions for implementing the function:

Context

```
class ChandrasekharSieve:
    """
    This is a class that uses the Chandrasekhar's Sieve
    method to find all prime numbers within the range
    """

    def __init__(self, n):
        """
        Initialize the ChandrasekharSieve class with the given limit.
        :param n: int, the upper limit for generating prime numbers
        """
        self.n = n
        self.primes = self.generate_primes()

    def generate_primes(self):
        pass
```

Instruction 1

```
def get_primes(self):
    """
    Get the list of generated prime numbers.
    :return: list, a list of prime numbers
    >>> cs = ChandrasekharSieve(20)
    >>> cs.get_primes()
    [2, 3, 5, 7, 11, 13, 17, 19]
    """
```

Instruction 2

```
def get_primes(self):
    """
    Access and return the list of already
    generated prime numbers. This function
    directly accesses 'self.primes' to get
    the list of stored primes.
    :return: list, a list of prime numbers
    """
```

Instruction 3

```
def get_primes(self):
    """
    Obtain and return the precomputed list
    of prime numbers that has been stored.
    The function should directly fetch the
    list of these previously computed primes.
    :return: list, a list of prime numbers
    """
```

14. In a few words, what is the task this function aims to implement? *

15. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

16. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

17. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 5:

Below are instructions for implementing the function:

Instruction 1

```
def double_the_difference(lst):
    """
    Given a list of numbers, return the sum
    of squares of the numbers
    in the list that are odd. Ignore numbers
    that are negative or not integers.

    double_the_difference([1, 3, 2, 0]) == 1 + 9 + 0 + 0 = 10
    double_the_difference([-1, -2, 0]) == 0
    double_the_difference([9, -2]) == 81
    double_the_difference([0]) == 0

    If the input list is empty, return 0.
    """
```

Instruction 2

```
def double_the_difference(lst):
    """
    Create the function 'double_the_difference'
    which takes a list 'lst'. The function
    calculates and returns the sum of the squares
    of numbers that are odd, greater than zero,
    and whole from 'lst'. It introduces an
    accumulator 'ans' at zero and iterates each
    'num' in 'lst', testing for oddness
    ('num % 2 == 1'), positivity ('num > 0'),
    and integrality (no '.' in 'str(num)').
    If a number meets all criteria, it is s
    quared ('num ** 2') and summed into 'ans'.
    The final 'ans' or 0 for an empty 'lst'
    is returned at the end.
    """
```

Instruction 3

```
def double_the_difference(lst):
    """
    Write a function named 'double_the_difference'
    that calculates the sum of the squares of
    numbers in a list that are odd, non-negative,
    and integers. The function should return 0
    for an empty list.
    """
```

18. In a few words, what is the task this function aims to implement? *

19. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

20. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

21. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 6:

Below are instructions for implementing the function:

Context

```
class DecryptionUtils:
    """
    This is a class that provides methods for decryption,
    including the Caesar cipher, Vigenere cipher, a
    nd Rail Fence cipher.
    """

    def __init__(self, key):
        """
        Initializes the decryption utility with a key.
        :param key: The key to use for decryption,str.
        """
        self.key = key

    def vigenere_decipher(self, ciphertext):
        pass

    def rail_fence_decipher(self, encrypted_text, rails):
        pass
```

Instruction 1

```
def caesar_decipher(self, ciphertext, shift):
    """
    Deciphers the given ciphertext using the Caesar cipher
    :param ciphertext: The ciphertext to decipher,str.
    :param shift: The shift to use for decryption,int.
    :return: The deciphered plaintext,str.
    >>> d = DecryptionUtils('key')
    >>> d.caesar_decipher('ifmmp', 1)
    'hello'
    """
```

Instruction 2

```
def caesar_decipher(self, ciphertext, shift):
    """
    Unravels the specified 'ciphertext' using a backward
    shift according to the Caesar cipher method.
    The process begins by establishing an empty string
    labeled 'plaintext' for accumulating decrypted symbols.
    As the process iterates over each symbol in 'ciphertext',
    it checks if the symbol is a letter (`char.isalpha()`).
    With affirmative, for uppercase letters (`char.isupper()`),
    it assigns an ASCII baseline of 65; for lowercase, 97 is used.
    It then processes to regain the original letter by deducting
    the ASCII base and 'shift', followed by applying modulo 26
    and adding the ASCII base before reconverting with `chr()`.
    Non-letter symbols are directly placed into 'plaintext'.
    Finally, it returns the completely formed 'plaintext'.
    :param ciphertext: The ciphertext to decipher,str.
    :param shift: The shift to use for decryption,int.
    :return: The deciphered plaintext,str.
    """
```

Instruction 3

```
def caesar_decipher(self, ciphertext, shift):
    """
    Using the Caesar cipher method, decode the provided
    'ciphertext' by reversing the characters using the
    designated 'shift'.
    :param ciphertext: The ciphertext to decipher,str.
    :param shift: The shift to use for decryption,int.
    :return: The deciphered plaintext,str.
    """
```

22. In a few words, what is the task this function aims to implement? *

23. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

24. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ◯ 1 min
- ◯ 5 min
- ◯ 10 min
- ◯ 20 min
- ◯ 40 min
- ◯ > 40 min

25. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 7:

Below are instructions for implementing the function:

Instruction 1

```
from typing import List, Tuple

def rolling_max(numbers: List[int]) -> List[int]:
    """
    From a given list of integers, generate a list of
    rolling maximum element found until given moment
    in the sequence.
    >>> rolling_max([1, 2, 3, 2, 3, 4, 2])
    [1, 2, 3, 3, 3, 4, 4]
    """
```

Instruction 2

```
from typing import List, Tuple

def rolling_max(numbers: List[int]) -> List[int]:
    """
    The function 'rolling_max' takes a list of integers,
    'numbers', as input. It is designed to return a new
    list where each index holds the maximum value from
    the beginning of 'numbers' to that particular index.
    A list comprehension is used here, iterating over
    'numbers' indices from 0 to the end. For each index
    'i', the function finds the maximum in 'numbers' up
    to that index via 'max(numbers[:(i+1)])'. This list
    of maximal values is generated and then returned.
    """
```

Instruction 3

```
from typing import List, Tuple

def rolling_max(numbers: List[int]) -> List[int]:
    """
    Formulate a function named 'rolling_max' that
    inputs a list of integers and delivers a list
    wherein each position i holds the maximum value
    starting from the beginning of the list to the
    position i.
    """
```

26. In a few words, what is the task this function aims to implement? *

27. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

28. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

29. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 8:

Below are instructions for implementing the function:

Context

```python
from datetime import datetime
class Chat:
    """
    This is a chat class with the functions of adding users,
    removing users, sending messages, and obtaining messages.
    """

    def __init__(self):
        """
        Initialize the Chat with an attribute users, which
        is an empty dictionary.
        """
        self.users = {}

    def add_user(self, username):
        pass

    def remove_user(self, username):
        pass

    def send_message(self, sender, receiver, message):
        pass
```

Instruction 1

```
def get_messages(self, username):
    """
    Get all the messages of a user from the Chat.
    :param username: The user's name, str.
    :return: A list of messages, each message is a dictionary
        with keys 'sender', 'receiver', 'message', 'timestamp'.
    >>> chat = Chat()
    >>> chat.users = {'John': [{'sender': 'John', 'receiver':
        'Mary', 'message': 'Hello', 'timestamp': '2023-01-01 00:00:00'}]}
    >>> chat.get_messages('John')
    [{'sender': 'John', 'receiver': 'Mary', 'message': 'Hello',
                            'timestamp': '2023-01-01 00:00:00'}]
    >>> chat.get_messages('Mary')
    []
    """
```

Instruction 2

```
def get_messages(self, username):
    """
    Retrieve all messages related to a specified user "username".
    If the user does not exist in the users dictionary, return
    an empty list. Otherwise, return a list of messages.
    Each message in the list should be a dictionary, comprising
    the keys 'sender', 'receiver', 'message', and 'timestamp'.
    This function checks the existence of the user and retrieves
    the corresponding messages directly from a dictionary attribute
    that holds user data.
    :param username: The user's name, str.
    :return: A list of messages, each message is a dictionary with
            keys 'sender', 'receiver', 'message', 'timestamp'.
    """
```

Instruction 3

```
def get_messages(self, username):
    """
    Obtain all messages for a particular user 'username'.
    If the user is not found in the users dictionary,
    the function should return an empty list. If found,
    return the messages as a list where each message
    is structured as a dictionary with 'sender',
    'receiver', 'message', and 'timestamp' keys.
    This function is designed to check if the user
    exists and to extract the applicable messages
    from a dictionary holding user data.
    :param username: The user's name, str.
    :return: A list of messages, each message is a dictionary
    with keys 'sender', 'receiver', 'message', 'timestamp'.
    """
```

30. In a few words, what is the task this function aims to implement? *

31. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

32. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

33. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 9:

Below are instructions for implementing the function:

Instruction 1

```
def vowels_count(s):
    """
    Write a function vowels_count which takes a string representing
    a word as input and returns the number of vowels in the string.
    Vowels in this case are 'a', 'e', 'i', 'o', 'u'. Here, 'y' is also a
    vowel, but only when it is at the end of the given word.

    Example:
    >>> vowels_count("abcde")
    2
    >>> vowels_count("ACEDY")
    3
    """
```

Instruction 2

```
def vowels_count(s):
    """
    Write a function named 'vowels_count' which counts the
    number of vowels in a given string. Vowels are defined
    as 'a', 'e', 'i', 'o', 'u', and 'y'. The character 'y'
    is considered a vowel only when it is the last character
    in the string.
    """
```

Instruction 3

```
def vowels_count(s):
    """
    Formulate a function named 'vowels_count' to count
    the vowels within a string. The vowels are
    'a', 'e', 'i', 'o', 'u', and 'y' where 'y'
    counts only if it's at the string's end.
    """
```

34. In a few words, what is the task this function aims to implement? *

35. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

36. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

37. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 10:

Below are instructions for implementing the function:

Context

```
import urllib.parse
class UrlPath:
    """
    The  class is a utility for encapsulating and manipulating
    the path component of a URL, including adding nodes,
    parsing path strings, and building path strings with
    optional encoding.
    """

    def __init__(self):
        """
        Initializes the UrlPath object with an empty list of segments
        and a flag indicating the presence of an end tag.
        """
        self.segments = []
        self.with_end_tag = False

    def add(self, segment):
        pass

    def parse(self, path, charset):
        pass


    @staticmethod
    def fix_path(path):

```

Instruction 1

```
def fix_path(path):
    """
    Fixes the given path string by removing
    leading and trailing slashes.
    :param path: str, the path string to fix.
    :return: str, the fixed path string.
    >>> url_path = UrlPath()
    >>> url_path.fix_path('/foo/bar/')
    'foo/bar'
    """
```

Instruction 2

```
def fix_path(path):
    """
    Revise the input string 'path' by stripping off
    the slashes from both the beginning and the end.
    Check if 'path' is nonexistent by utilizing
    'if not path', and respond by returning an empty
    string (''). Conversely, if there is content,
    strip the beginning and ending slashes using
    'path.strip('/')', thereby normalizing the 'path'.
    Subsequently, return the revised path string.
    :param path: str, the path string to fix.
    :return: str, the fixed path string.
    """
```

Instruction 3

```
def fix_path(path):
    """
    For the provided path string 'path', remove
    any leading and trailing slashes. The function
    should first verify if 'path' is non-empty.
    In case it's empty, simply return an empty string.
    If non-empty, remove the slashes at both ends
    and return the modified path string.
    :param path: str, the path string to fix.
    :return: str, the fixed path string.
    """
```

38. In a few words, what is the task this function aims to implement? *

39. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

40. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

41. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 11:

Below are instructions for implementing the function

Instruction 1

```
def choose_num(x, y):
    """
    This function takes two positive numbers x and y and returns the
    biggest even integer number that is in the range [x, y] inclusive. If
    there's no such number, then the function should return -1.

    For example:
    choose_num(12, 15) = 14
    choose_num(13, 12) = -1
    """
```

Instruction 2

```
def choose_num(x, y):
    """
    Construct a function 'choose_num' that receives 'x' and 'y' as inputs,
    both being positive integers. The function initially determines if
    'x' exceeds 'y', returning -1 if this is true. If 'x' and 'y' are equal,
    the function checks the parity of 'y' using 'y % 2 == 0'. If 'y' is even,
    it is returned, else -1 is returned. Lastly, if 'x' is less than 'y',
    the function outputs 'y' directly if 'y' is even, otherwise it gives
    'y - 1' to make it even.
    """
```

Instruction 3

```
def choose_num(x, y):
    """
    Define the function 'choose_num' with parameters 'x' and 'y',
    where both are positive integers. Initially, it assesses whether
    'x' surpasses 'y'. If so, the return value is -1. In the case
    where 'x' equals 'y', it then examines if 'y' is divisible evenly
    by 2 (using 'y % 2 == 0'). It returns 'y' if it's even, otherwise,
    it gives back -1. If 'x' is smaller than 'y', the function yields
    'y' if even; otherwise, 'y - 1' is returned to adjust it to an even
    number.
    """
```

42. In a few words, what is the task this function aims to implement? *

43. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- [ ] Very Easy
- [ ] Easy
- [ ] Not Easy Nor Difficult
- [ ] Difficult
- [ ] Very Difficult

44. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- [ ] 1 min
- [ ] 5 min
- [ ] 10 min
- [ ] 20 min
- [ ] 40 min
- [ ] > 40 min

45. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 12:

Below are instructions for implementing the function:

Context

```
class SQLQueryBuilder:
    """
    This class provides to build SQL queries, including SELECT,
    INSERT, UPDATE, and DELETE statements.
    """

    def select(table, columns='*', where=None):
        pass

    @staticmethod
    def insert(table, data):
        pass

    @staticmethod
    def delete(table, where=None):
        pass


    @staticmethod
    def update(table, data, where=None):

```

Instruction 1

```
def update(table, data, where=None):
    """
    Generate the UPDATE SQL statement from the given parameters.
    :param table: str, the table that will be excuted with UPDATE
                                        operation in database
    :param data: dict, the key and value in SQL update statement
    :param where: dict, {key1: value1, key2: value2 ...}.
    The query condition.
    >>> SQLQueryBuilder.update('table1',
            {'name': 'Test2', 'age': 15}, where = {'name':'Test'})
    "UPDATE table1 SET name='Test2', age='15' WHERE name='Test'"
    """
```

Instruction 2

```
def update(table, data, where=None):
    """
    Create an UPDATE SQL statement using the specified 'table', 'data',
    and optional 'where' parameters. The procedure prepares a query for
    updating the named 'table', transforming 'data' into a comma-separated
    sequence of 'key=value' pairs using list comprehension. If 'where' is
    not None, the function includes a 'WHERE' clause in the query.
    This clause uses another list comprehension to create 'key=value'
    strings for 'where', connecting them with 'AND' to delineate which
    records should be affected.
    :param table: str, the table that will be excuted with UPDATE operation
                                                                in database
    :param data: dict, the key and value in SQL update statement
    :param where: dict, {key1: value1, key2: value2 ...}. The query condition.
    """
```

Instruction 3

```
def update(table, data, where=None):
    """
    Generate an SQL UPDATE statement based on the given 'table', 'data',
    and 'where' parameters. The function formulates an update query for
    the specified 'table' by converting 'data' key-value pairs into
    comma-ranked SQL assignments. With an optional 'where' dictionary,
    the query additionally gains a 'WHERE' clause that strings together
    all conditions using 'AND', pinpointing exactly which records need modifying.
    :param table: str, the table that will be excuted with UPDATE operation in database
    :param data: dict, the key and value in SQL update statement
    :param where: dict, {key1: value1, key2: value2 ...}. The query condition.
    """
```

46. In a few words, what is the task this function aims to implement? *

_____

47. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

48. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- [ ] 1 min
- [ ] 5 min
- [ ] 10 min
- [ ] 20 min
- [ ] 40 min
- [ ] > 40 min

49. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Demographics Information

50. What is your highest education qualification? *

Une seule réponse possible.

- [ ] Ph.D.
- [ ] Master's Degree
- [ ] Bachelor Degree
- [ ] Autre : _____

51. What is your field of work? (Engineer, Researcher, Developer, etc.) *

_____

52. How many years of development experience do you have? *

Une seule réponse possible.

- [ ] Less than 5
- [ ] Between 5 and 10
- [ ] More than 10

53. Thank you for completing the survey! Any feedback you would like to provide?

_____

_____

_____

_____

_____